SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 06 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Experiments

Code of Conduct:

I M. Pranay (192525382) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M. Pranay*

Annexure A

Short Answer Test

1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

6. Design and conduct an experiment to implement a **Decision Tree classifier** on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the

impact of tree depth and pruning on overfitting, and compare results across different configurations.

7. Perform an experiment using the **K-Nearest Neighbors (KNN)** algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan) to observe their effect on model performance. Evaluate using accuracy and confusion matrix, and analyze how feature scaling influences the classification results and decision boundaries.

8. Implement a **Support Vector Machine (SVM)** model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

9. Design an experiment to implement a **clustering algorithm such as K-Means** on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

10. Conduct an experiment using a **logistic regression model** for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

11. Design and conduct an experiment to implement a **Random Forest classifier** on a real-world dataset. Train the model with varying numbers of trees and maximum depth. Evaluate performance using accuracy, precision, recall, and F1-score. Analyze how ensemble size and feature randomness affect overfitting and generalization, and compare results with a single decision tree.

12. Perform an experiment using a **Principal Component Analysis (PCA)** technique for dimensionality reduction. Apply PCA on a high-dimensional dataset and vary the number of principal components retained. Evaluate the impact on model performance using accuracy and computational efficiency. Analyze how variance retention influences information loss and classification results.

13. Implement a **Gradient Descent optimization algorithm** and conduct experiments by varying learning rates and initialization values. Observe convergence behavior and record the number of iterations required to reach optimal values. Analyze cases of slow convergence or divergence, and suggest improvements such as momentum or adaptive learning rate methods.

14. Design an experiment to implement a **Neural Network classifier** for a classification task. Vary the number of hidden layers, neurons, and activation functions. Evaluate the model using accuracy and loss curves, and analyze how network architecture affects performance, convergence, and overfitting.

15. Conduct an experiment using a **hierarchical clustering algorithm** on an unlabeled dataset. Compare different linkage methods such as single, complete, and average linkage. Evaluate clustering quality using dendrogram analysis and silhouette score. Analyze how linkage criteria influence cluster formation and interpretability of results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

1. Implement a Naïve Bayes Classifier on a Real-World Dataset

Aim: To implement a Naïve Bayes classifier and evaluate its performance using accuracy, precision, and recall.

Algorithm:

1. Import the required libraries such as Pandas and Scikit-learn.
2. Load a real-world dataset.
3. Divide the dataset into training and testing data.
4. Train the Naïve Bayes classifier.
5. Predict the output using test data.
6. Calculate accuracy, precision, and recall.
7. Compare results with and without Laplace smoothing.

Performance Metrics:

- Accuracy: Measures the overall correct predictions.
- Precision: Measures how many predicted positive values are correct.
- Recall: Measures how many actual positive values are identified correctly.

Analysis:

- Naïve Bayes assumes that all features are independent.
- Feature independence may affect classification accuracy.
- Laplace smoothing avoids zero probability problems.
- Smoothing improves probability estimation when some values are missing.

Result: The Naïve Bayes classifier is implemented successfully, and its performance is evaluated using accuracy, precision, and recall.

2. Maximum Likelihood Estimation (MLE) and Bayesian Estimation

Aim: To estimate the parameters of a probability distribution using MLE and Bayesian estimation.

Algorithm:

1. Import the required libraries.
2. Load a suitable dataset.
3. Calculate the mean and variance of the dataset.
4. Apply Maximum Likelihood Estimation (MLE).
5. Apply Bayesian estimation using prior assumptions.
6. Compare both estimation methods.
7. Observe the results for small and large sample sizes.

Formulas:

Mean:

$$\mu = \frac{\sum x_i}{n}$$

Variance:

$$\sigma^2 = \frac{\sum (x_i - \mu)^2}{n}$$

Analysis:

- MLE estimates parameters using the observed data.
- Bayesian estimation combines prior knowledge with observed data.
- For small datasets, prior assumptions have a greater influence.
- For large datasets, observed data has more influence.
- MLE is generally more reliable with sufficient data.

Result: MLE and Bayesian estimation are performed successfully, and their parameter estimates are compared.

3. Expectation-Maximization (EM) Algorithm for Mixture Models

Aim: To implement the EM algorithm and study its convergence behaviour for different cluster numbers and initializations.

Algorithm:

1. Import the required libraries.
2. Load the dataset.
3. Select the number of clusters.
4. Initialize the model parameters.
5. Perform the Expectation (E) step to calculate probabilities.
6. Perform the Maximization (M) step to update parameters.
7. Repeat both steps until convergence.
8. Compare convergence speed and final likelihood values.

Analysis:

- EM is used to estimate parameters when data contains hidden variables.
- The E-step estimates the probability of data belonging to each cluster.
- The M-step updates the model parameters.
- Different initializations may produce different results.
- The algorithm may converge to a local optimum instead of the global optimum.

Methods to Improve Performance:

- Use multiple initializations.
- Select suitable cluster numbers.

- Increase the number of iterations.
- Use proper initial parameter values.

Result: The EM algorithm is implemented successfully, and its convergence behaviour is analysed.

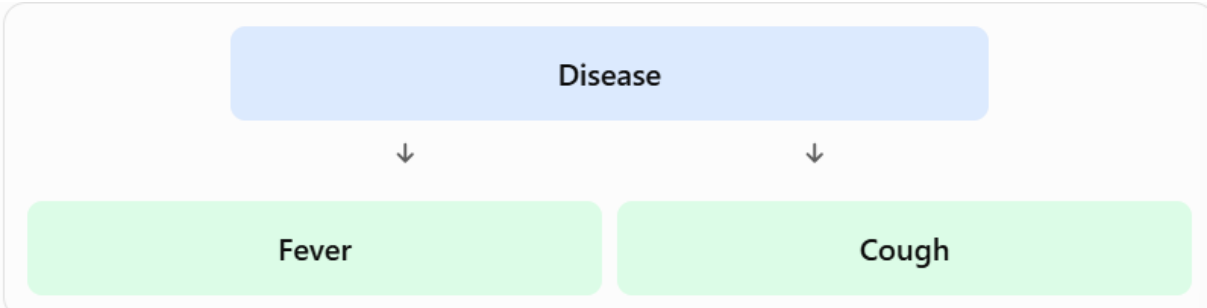
4. Bayesian Belief Network and Probabilistic Inference

Aim: To construct a Bayesian Belief Network and perform inference under different evidence conditions.

Concept:

A Bayesian Belief Network is a graphical model that represents relationships between variables using nodes and directed edges.

Example:



Algorithm:

1. Identify the variables in the problem.
2. Create nodes for each variable.
3. Connect nodes using directed edges.
4. Assign conditional probabilities.
5. Provide evidence for different conditions.
6. Calculate posterior probabilities.
7. Modify probabilities and observe the changes.

Analysis:

- Nodes represent random variables.
- Edges represent dependencies between variables.
- Conditional probability tables describe relationships.
- Evidence changes the posterior probability.
- Bayesian networks help manage uncertainty in decision-making.

Result: A Bayesian Belief Network is constructed successfully, and probabilistic inference is performed under different evidence conditions.

5. Concept Learning Using Candidate Elimination Algorithm

Aim: To implement the Candidate Elimination algorithm and analyse the effect of hypothesis space size on learning performance.

Concept:

Concept learning is the process of learning a target concept from training examples.

Candidate Elimination maintains two boundaries:

- S – Specific Boundary: Most specific hypothesis.
- G – General Boundary: Most general hypothesis.

Algorithm:

1. Initialize the specific boundary (S).
2. Initialize the general boundary (G).
3. Read the training examples.
4. For positive examples, generalize S.
5. For negative examples, specialize G.
6. Remove inconsistent hypotheses.
7. Continue until all training examples are processed.
8. Observe the final hypothesis boundaries.

Analysis:

- Increasing the number of attributes increases the hypothesis space.
- More training examples can improve learning.
- A finite hypothesis space is easier to search.
- An infinite hypothesis space requires more complex learning.
- Sample complexity affects the number of examples needed for learning.
- The mistake bound model measures the maximum number of prediction errors.

Result: The Candidate Elimination algorithm is implemented successfully, and the effect of hypothesis space size on learning performance is analysed.